

E-JAVA LAB ASSIGNMENT-2

30/01/2026

1. Online Shopping Cart System

Design a Product class with attributes like productId, productName, price, and quantity.

Tasks:

- Use **constructors** (default and parameterized)
- Use **method overloading** to calculate total price with and without discount
- Use this keyword to resolve variable conflicts
- Restrict direct access to price using **access control**

```
class Product {  
  
    // Attributes  
    private int productId;  
    private String productName;  
    private double price; // Restricted access  
    private int quantity;  
  
    // Default Constructor  
    public Product() {  
        this.productId = 0;  
        this.productName = "Unknown";  
        this.price = 0.0;  
        this.quantity = 0;  
    }  
  
    // Parameterized Constructor  
    public Product(int productId, String productName, double price, int quantity) {  
        this.productId = productId; // using this keyword  
        this.productName = productName; // resolves variable conflict  
        this.price = price;  
        this.quantity = quantity;  
    }  
  
    // Getter for price (controlled access)  
    public double getPrice() {  
        return this.price;  
    }  
  
    // Method Overloading: Calculate total price without discount  
    public double calculateTotal() {
```

```

        return this.price * this.quantity;
    }

    // Method Overloading: Calculate total price with discount
    public double calculateTotal(double discountPercentage) {
        double total = this.price * this.quantity;
        double discountAmount = total * discountPercentage / 100;
        return total - discountAmount;
    }

    // Display product details
    public void display() {
        System.out.println("Product ID   : " + productId);
        System.out.println("Product Name : " + productName);
        System.out.println("Price      : ₹" + price);
        System.out.println("Quantity   : " + quantity);
    }
}

public class ShoppingCartSystem {
    public static void main(String[] args) {

        // Using parameterized constructor
        Product p1 = new Product(101, "Headphones", 1500.0, 2);

        p1.display();

        System.out.println("Total Price (No Discount): ₹" + p1.calculateTotal());
        System.out.println("Total Price (10% Discount): ₹" + p1.calculateTotal(10));
    }
}

```

Output:

```

Product ID   : 101
Product Name : Headphones
Price      : ₹1500.0
Quantity   : 2
Total Price (No Discount): ₹3000.0
Total Price (10% Discount): ₹2700.0

```

2. Bank Account Management System

Create a BankAccount class to represent customer accounts.

Requirements:

- Use **private variables** for account details

- Use **constructors** for account initialization
- Use **static variable** to store bank name (common for all accounts)
- Use **final variable** for minimum balance
- Implement methods for deposit and withdrawal

```
class BankAccount {

    // Private variables (Encapsulation)
    private int accountNumber;
    private String accountHolderName;
    private double balance;

    // Static variable (common for all accounts)
    static String bankName = "National Bank";

    // Final variable (constant minimum balance)
    final double MIN_BALANCE = 1000.0;

    // Constructor
    public BankAccount(int accountNumber, String accountHolderName, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
    }

    // Deposit method
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Amount Deposited: ₹" + amount);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    // Withdraw method
    public void withdraw(double amount) {
        if (amount > 0 && (balance - amount) >= MIN_BALANCE) {
            balance -= amount;
            System.out.println("Amount Withdrawn: ₹" + amount);
        } else {
            System.out.println("Withdrawal denied! Minimum balance must be maintained.");
        }
    }

    // Display account details
    public void display() {
        System.out.println("Bank Name    : " + bankName);
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Account Holder : " + accountHolderName);
    }
}
```

```

        System.out.println("Balance      : ₹" + balance);
    }
}

public class BankAccountManagement {
    public static void main(String[] args) {

        BankAccount acc1 = new BankAccount(1001, "Rahul", 5000);

        acc1.display();

        acc1.deposit(2000);
        acc1.withdraw(4000);

        acc1.display();
    }
}

```

Output:

```

Bank Name      : National Bank
Account Number : 1001
Account Holder : Rahul
Balance       : ₹5000.0
Amount Deposited: ₹2000.0
Amount Withdrawn: ₹4000.0
Bank Name      : National Bank
Account Number : 1001
Account Holder : Rahul
Balance       : ₹3000.0

```

3. Employee Payroll Processing

Develop an Employee class to calculate salary.

Tasks:

- Use **method overloading** to calculate salary for permanent and contract employees
- Use **access specifiers** properly
- Use static method to display company policies
- Use **final** method to prevent modification of tax calculation logic

```

class Employee {

    // Private variables (Encapsulation)
    private int empId;

```

```

private String empName;
private double basicSalary;

// Constructor
public Employee(int empId, String empName, double basicSalary) {
    this.empId = empId;
    this.empName = empName;
    this.basicSalary = basicSalary;
}

// Method Overloading: Salary for Permanent Employee
public double calculateSalary(double hra, double da) {
    double gross = basicSalary + hra + da;
    return gross - calculateTax(gross);
}

// Method Overloading: Salary for Contract Employee
public double calculateSalary(int workingDays, double perDayAmount) {
    double gross = workingDays * perDayAmount;
    return gross - calculateTax(gross);
}

// Final method (Tax logic cannot be overridden)
public final double calculateTax(double amount) {
    return amount * 0.10; // 10% tax
}

// Static method (Common company policy)
public static void companyPolicy() {
    System.out.println("Company Policy: 10% tax deduction applicable to all employees.");
}

// Display employee details
public void display() {
    System.out.println("Employee ID : " + empId);
    System.out.println("Employee Name : " + empName);
    System.out.println("Basic Salary : ₹" + basicSalary);
}
}

public class PayrollProcessing {
    public static void main(String[] args) {

        Employee.companyPolicy(); // Static method call

        Employee emp1 = new Employee(101, "Anita", 30000);

        emp1.display();

        double permanentSalary = emp1.calculateSalary(5000, 3000);
    }
}

```

```

        System.out.println("Net Salary (Permanent): ₹" + permanentSalary);

        double contractSalary = emp1.calculateSalary(20, 1500);
        System.out.println("Net Salary (Contract): ₹" + contractSalary);
    }
}

```

Output:

Company Policy: 10% tax deduction applicable to all employees.

Employee ID : 101

Employee Name : Anita

Basic Salary : ₹30000.0

Net Salary (Permanent): ₹34200.0

Net Salary (Contract): ₹27000.0

4. College Course Registration System

Create a Course class and a Student class.

Requirements:

- Demonstrate **object reference assignment**
- Use **array of objects**
- Implement constructors to initialize student data
- Use this keyword for constructor chaining
- If the student is not registered for any course, display the message "Course not registered"

```

class Course {
    String courseId;
    String courseName;

    // Constructor
    public Course(String courseId, String courseName) {
        this.courseId = courseId;
        this.courseName = courseName;
    }

    public void displayCourse() {
        System.out.println("Course ID : " + courseId);
        System.out.println("Course Name : " + courseName);
    }
}

class Student {

```

```

private int studentId;
private String studentName;
private Course course; // Object reference

// Default Constructor (Constructor Chaining)
public Student() {
    this(0, "Unknown", null); // calling parameterized constructor
}

// Parameterized Constructor
public Student(int studentId, String studentName, Course course) {
    this.studentId = studentId; // using this keyword
    this.studentName = studentName;
    this.course = course;
}

public void displayStudent() {
    System.out.println("Student ID : " + studentId);
    System.out.println("Student Name : " + studentName);

    if (course != null) {
        course.displayCourse();
    } else {
        System.out.println("Course not registered");
    }
}
}

public class CollegeRegistrationSystem {
    public static void main(String[] args) {

        // Creating Course object
        Course c1 = new Course("CS101", "Data Structures");

        // Array of Student objects
        Student[] students = new Student[3];

        // Object reference assignment
        students[0] = new Student(1, "Rahul", c1); // Registered
        students[1] = new Student(2, "Anita", null); // Not registered
        students[2] = new Student(); // Using default constructor

        // Display student details
        for (Student s : students) {
            s.displayStudent();
            System.out.println();
        }
    }
}

```

Output:

Student ID : 1
Student Name : Rahul
Course ID : CS101
Course Name : Data Structures

Student ID : 2
Student Name : Anita
Course not registered

Student ID : 0
Student Name : Unknown
Course not registered

5. Ride Booking Application (Mini Ola/Uber)

Design a Ride class.

Tasks:

- Overload methods to calculate fare based on distance, time, and surge pricing
- Use **static variable** for base fare
- Use **access control** to protect fare calculation logic
- Demonstrate object references pointing to different rides

```
class Ride {  
  
    private String rideId;  
    private double distance; // in km  
    private int time;        // in minutes  
  
    // Static variable (common base fare)  
    static double baseFare = 50.0;  
  
    // Constructor  
    public Ride(String rideId, double distance, int time) {  
        this.rideId = rideId;  
        this.distance = distance;  
        this.time = time;  
    }  
  
    // Overloaded method: Fare based only on distance  
    public double calculateFare(double ratePerKm) {  
        return computeFare(distance * ratePerKm);  
    }  
}
```

```

// Overloaded method: Fare based on distance and time
public double calculateFare(double ratePerKm, double ratePerMinute) {
    double total = (distance * ratePerKm) + (time * ratePerMinute);
    return computeFare(total);
}

// Overloaded method: Fare with surge pricing
public double calculateFare(double ratePerKm, double ratePerMinute, double
surgeMultiplier) {
    double total = ((distance * ratePerKm) + (time * ratePerMinute)) * surgeMultiplier;
    return computeFare(total);
}

// Private method (Protected fare calculation logic)
private double computeFare(double amount) {
    return baseFare + amount;
}

public void display() {
    System.out.println("Ride ID : " + rideId);
    System.out.println("Distance : " + distance + " km");
    System.out.println("Time    : " + time + " minutes");
}
}

public class RideBookingApp {
    public static void main(String[] args) {

        // Object references pointing to different rides
        Ride ride1 = new Ride("R101", 10, 15);
        Ride ride2 = new Ride("R102", 5, 10);

        ride1.display();
        System.out.println("Fare (Distance only): ₹" + ride1.calculateFare(12));
        System.out.println("Fare (Distance + Time): ₹" + ride1.calculateFare(12, 2));
        System.out.println("Fare (With Surge 1.5x): ₹" + ride1.calculateFare(12, 2, 1.5));
        System.out.println();

        ride2.display();
        System.out.println("Fare (Distance + Time): ₹" + ride2.calculateFare(12, 2));
    }
}

```

Output:

```

Ride ID : R101
Distance : 10.0 km
Time    : 15 minutes
Fare (Distance only): ₹170.0
Fare (Distance + Time): ₹200.0
Fare (With Surge 1.5x): ₹275.0

```

Ride ID : R102
Distance : 5.0 km
Time : 10 minutes
Fare (Distance + Time): ₹130.0

6. Library Management System

Create Book and Library classes.

Requirements:

- Use **nested classes** to represent book categories
- Use **static nested class** for library rules
- Restrict direct access to book availability
- Use constructors to initialize book details

```
class Library {  
  
    // Static Nested Class (Library Rules)  
    static class LibraryRules {  
        public static void showRules() {  
            System.out.println("Library Rules:");  
            System.out.println("1. Return books within 14 days.");  
            System.out.println("2. Maintain silence.");  
            System.out.println("3. Fine applicable for late return.");  
        }  
    }  
}  
  
// Inner Class representing Book  
class Book {  
  
    private String bookId;  
    private String title;  
    private boolean available; // Restricted access  
  
    // Nested class to represent Book Category  
    class Category {  
        private String categoryName;  
  
        public Category(String categoryName) {  
            this.categoryName = categoryName;  
        }  
  
        public void displayCategory() {  
            System.out.println("Category : " + categoryName);  
        }  
    }  
}
```

```

    }

    // Constructor
    public Book(String bookId, String title, boolean available) {
        this.bookId = bookId;
        this.title = title;
        this.available = available;
    }

    // Getter method (controlled access)
    public boolean isAvailable() {
        return available;
    }

    // Method to issue book
    public void issueBook() {
        if (available) {
            available = false;
            System.out.println("Book issued successfully.");
        } else {
            System.out.println("Book not available.");
        }
    }

    // Display book details
    public void displayBook() {
        System.out.println("Book ID   : " + bookId);
        System.out.println("Title    : " + title);
        System.out.println("Available : " + available);
    }
}

public class LibraryManagementSystem {
    public static void main(String[] args) {

        // Access static nested class
        Library.LibraryRules.showRules();
        System.out.println();

        Library library = new Library();

        // Creating Book object
        Library.Book book1 = library.new Book("B101", "Java Programming", true);

        // Creating Category object
        Library.Book.Category category = book1.new Category("Technology");

        book1.displayBook();
        category.displayCategory();
    }
}

```

```
        book1.issueBook();
        book1.displayBook();
    }
}
```

Output:

Library Rules:

1. Return books within 14 days.
2. Maintain silence.
3. Fine applicable for late return.

Book ID : B101

Title : Java Programming

Available : true

Category : Technology

Book issued successfully.

Book ID : B101

Title : Java Programming

Available : false

7. Hospital Patient Registration System

Design a Patient class.

Tasks:

- Use **final class** to prevent inheritance
- Use **constructor overloading** for different patient types
- Use static variable to maintain hospital name
- Apply proper access control for medical data

// Final class (cannot be inherited)

```
final class Patient {
```

```
    // Static variable (common hospital name)
```

```
    static String hospitalName = "City Care Hospital";
```

```
    // Private variables (medical data protected)
```

```
    private int patientId;
```

```
    private String patientName;
```

```
    private int age;
```

```
    private String medicalHistory;
```

```
    // Constructor for regular patient
```

```
    public Patient(int patientId, String patientName, int age) {
        this.patientId = patientId;
```

```

        this.patientName = patientName;
        this.age = age;
        this.medicalHistory = "Not Provided";
    }

    // Overloaded constructor for critical patient (with medical history)
    public Patient(int patientId, String patientName, int age, String medicalHistory) {
        this.patientId = patientId;
        this.patientName = patientName;
        this.age = age;
        this.medicalHistory = medicalHistory;
    }

    // Getter for medical data (controlled access)
    public String getMedicalHistory() {
        return medicalHistory;
    }

    // Display patient details
    public void display() {
        System.out.println("Hospital Name : " + hospitalName);
        System.out.println("Patient ID   : " + patientId);
        System.out.println("Patient Name : " + patientName);
        System.out.println("Age       : " + age);
    }
}

public class HospitalRegistrationSystem {
    public static void main(String[] args) {

        // Creating different types of patients
        Patient p1 = new Patient(101, "Rahul", 30);
        Patient p2 = new Patient(102, "Anita", 45, "Diabetes & Hypertension");

        p1.display();
        System.out.println("Medical History: " + p1.getMedicalHistory());
        System.out.println();

        p2.display();
        System.out.println("Medical History: " + p2.getMedicalHistory());
    }
}

```

Output:

```

Hospital Name : City Care Hospital
Patient ID   : 101
Patient Name : Rahul
Age       : 30
Medical History: Not Provided

```

Hospital Name : City Care Hospital
Patient ID : 102
Patient Name : Anita
Age : 45
Medical History: Diabetes & Hypertension

8. Online Food Delivery Order System

Create an Order class.

Requirements:

- Use **inner class** to represent OrderItem
- Overload methods to calculate bill with tax and delivery charges
- Use this keyword to differentiate instance and local variables
- Use static method to show restaurant details

```
class Order {  
  
    private int orderId;  
    private String customerName;  
  
    // Constructor  
    public Order(int orderId, String customerName) {  
        this.orderId = orderId;    // using this keyword  
        this.customerName = customerName;  
    }  
  
    // Inner class to represent Order Item  
    class OrderItem {  
        private String itemName;  
        private double price;  
        private int quantity;  
  
        public OrderItem(String itemName, double price, int quantity) {  
            this.itemName = itemName;    // differentiating instance & local variable  
            this.price = price;  
            this.quantity = quantity;  
        }  
  
        public double getItemTotal() {  
            return this.price * this.quantity;  
        }  
  
        public void displayItem() {  
            System.out.println("Item Name : " + itemName);  
            System.out.println("Price : ₹" + price);  
        }  
    }  
}
```

```

        System.out.println("Quantity : " + quantity);
    }
}

// Overloaded method: Calculate bill without extra charges
public double calculateBill(double amount) {
    return amount;
}

// Overloaded method: Calculate bill with tax and delivery
public double calculateBill(double amount, double taxPercent, double deliveryCharge) {
    double tax = amount * taxPercent / 100;
    return amount + tax + deliveryCharge;
}

// Static method to show restaurant details
public static void showRestaurantDetails() {
    System.out.println("Restaurant: Spice Garden");
    System.out.println("Location : City Center");
    System.out.println("Contact : 9876543210");
}
}

public class FoodDeliverySystem {
    public static void main(String[] args) {

        Order.showRestaurantDetails();
        System.out.println();

        Order order = new Order(101, "Rahul");

        // Creating inner class object
        Order.OrderItem item1 = order.new OrderItem("Pizza", 250, 2);

        item1.displayItem();

        double baseAmount = item1.getItemTotal();
        System.out.println("Base Bill: ₹" + order.calculateBill(baseAmount));

        double finalAmount = order.calculateBill(baseAmount, 5, 40);
        System.out.println("Final Bill (with Tax & Delivery): ₹" + finalAmount);
    }
}

```

Output:

```

Restaurant: Spice Garden
Location : City Center
Contact : 9876543210

```

```

Item Name : Pizza

```

Price : ₹250.0
Quantity : 2
Base Bill: ₹500.0
Final Bill (with Tax & Delivery): ₹565.0

9. Vehicle Rental System

Design a Vehicle class.

Tasks:

- Use **method overloading** for rent calculation (hourly, daily)
- Use **final variable** for registration number
- Use access control to protect internal logic
- Demonstrate multiple object references

```
class Vehicle {  
  
    // Final variable (cannot be changed once assigned)  
    private final String registrationNumber;  
  
    private String vehicleType;  
    private double hourlyRate;  
    private double dailyRate;  
  
    // Constructor  
    public Vehicle(String registrationNumber, String vehicleType,  
                    double hourlyRate, double dailyRate) {  
        this.registrationNumber = registrationNumber;  
        this.vehicleType = vehicleType;  
        this.hourlyRate = hourlyRate;  
        this.dailyRate = dailyRate;  
    }  
  
    // Method Overloading: Rent calculation hourly  
    public double calculateRent(int hours) {  
        return computeRent(hours * hourlyRate);  
    }  
  
    // Method Overloading: Rent calculation daily  
    public double calculateRent(int days, boolean isDaily) {  
        if (isDaily) {  
            return computeRent(days * dailyRate);  
        } else {  
            return 0;  
        }  
    }  
}
```

```

// Private method (protected internal logic)
private double computeRent(double amount) {
    return amount;
}

// Display details
public void display() {
    System.out.println("Registration No : " + registrationNumber);
    System.out.println("Vehicle Type   : " + vehicleType);
}
}

public class VehicleRentalSystem {
    public static void main(String[] args) {

        // Multiple object references
        Vehicle v1 = new Vehicle("KA01AB1234", "Car", 200, 1500);
        Vehicle v2 = new Vehicle("KA05XY5678", "Bike", 100, 700);

        v1.display();
        System.out.println("Hourly Rent (5 hrs): ₹" + v1.calculateRent(5));
        System.out.println("Daily Rent (2 days): ₹" + v1.calculateRent(2, true));
        System.out.println();

        v2.display();
        System.out.println("Hourly Rent (3 hrs): ₹" + v2.calculateRent(3));
    }
}

```

Output:

```

Registration No : KA01AB1234
Vehicle Type   : Car
Hourly Rent (5 hrs): ₹1000.0
Daily Rent (2 days): ₹3000.0

```

```

Registration No : KA05XY5678
Vehicle Type   : Bike
Hourly Rent (3 hrs): ₹300.0

```

10. E-Wallet Application

Create a Wallet class.

Requirements:

- Use **static variable** to track total wallets created

- Use constructors for wallet initialization
- Use access specifiers for balance
- Prevent modification of wallet ID using final

```
class Wallet {

    // Static variable to track total wallets created
    private static int totalWallets = 0;

    // Final variable (cannot be modified after initialization)
    private final String walletId;

    // Private variable (balance protected)
    private double balance;

    // Constructor
    public Wallet(String walletId, double initialBalance) {
        this.walletId = walletId;    // using final variable
        this.balance = initialBalance;
        totalWallets++;              // increment count
    }

    // Deposit method
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Amount Deposited: ₹" + amount);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    // Withdraw method
    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Amount Withdrawn: ₹" + amount);
        } else {
            System.out.println("Insufficient balance.");
        }
    }

    // Getter for balance (controlled access)
    public double getBalance() {
        return balance;
    }

    // Static method to show total wallets created
    public static int getTotalWallets() {
        return totalWallets;
    }
}
```

```

    }

    // Display wallet details
    public void display() {
        System.out.println("Wallet ID : " + walletId);
        System.out.println("Balance   : ₹" + balance);
    }
}

public class EWalletApplication {
    public static void main(String[] args) {

        Wallet w1 = new Wallet("W101", 1000);
        Wallet w2 = new Wallet("W102", 500);

        w1.display();
        w1.deposit(300);
        w1.withdraw(200);
        System.out.println("Updated Balance: ₹" + w1.getBalance());
        System.out.println();

        w2.display();
        System.out.println();

        System.out.println("Total Wallets Created: " + Wallet.getTotalWallets());
    }
}

```

Output:

```

Wallet ID : W101
Balance   : ₹1000.0
Amount Deposited: ₹300.0
Amount Withdrawn: ₹200.0
Updated Balance: ₹1100.0

```

```

Wallet ID : W102
Balance   : ₹500.0

```

```

Total Wallets Created: 2

```

11. Smart Electricity Billing System

Develop a ElectricityBill class.

Tasks:

- Use constructor to initialize consumer details

- Overload methods to calculate bill for domestic and commercial users
- Use static variable for tariff rate
- Apply access control for meter readings

```
class ElectricityBill {

    // Static variable for tariff rate (common base charge per unit)
    private static double tariffRate = 5.0;

    // Private variables (access control for meter readings)
    private int consumerId;
    private String consumerName;
    private int meterReading;

    // Constructor to initialize consumer details
    public ElectricityBill(int consumerId, String consumerName, int meterReading) {
        this.consumerId = consumerId;
        this.consumerName = consumerName;
        this.meterReading = meterReading;
    }

    // Overloaded method: Domestic user bill calculation
    public double calculateBill(int unitsConsumed) {
        return computeBill(unitsConsumed * tariffRate);
    }

    // Overloaded method: Commercial user bill calculation
    public double calculateBill(int unitsConsumed, double commercialRate) {
        return computeBill(unitsConsumed * commercialRate);
    }

    // Private method to protect internal billing logic
    private double computeBill(double amount) {
        return amount;
    }

    // Getter for meter reading (controlled access)
    public int getMeterReading() {
        return meterReading;
    }

    // Display consumer details
    public void display() {
        System.out.println("Consumer ID : " + consumerId);
        System.out.println("Consumer Name : " + consumerName);
        System.out.println("Meter Reading : " + meterReading);
    }
}

public class SmartElectricityBillingSystem {
```

```

public static void main(String[] args) {

    ElectricityBill domesticUser =
        new ElectricityBill(101, "Rahul", 350);

    ElectricityBill commercialUser =
        new ElectricityBill(102, "Anita", 500);

    domesticUser.display();
    System.out.println("Domestic Bill: ₹" +
        domesticUser.calculateBill(350));
    System.out.println();

    commercialUser.display();
    System.out.println("Commercial Bill: ₹" +
        commercialUser.calculateBill(500, 8.0));
}
}

```

Output:

```

Consumer ID : 101
Consumer Name : Rahul
Meter Reading : 350
Domestic Bill: ₹1750.0

```

```

Consumer ID : 102
Consumer Name : Anita
Meter Reading : 500
Commercial Bill: ₹4000.0

```

12. Student Attendance Tracking System

Create Student and Attendance classes.

Requirements:

- Use **inner class** for attendance details
- Use **this** keyword to pass current object
- Use static method to display attendance policy
- Demonstrate object reference reassignment

```

class Student {

    private int studentId;
    private String studentName;

    // Constructor

```

```

public Student(int studentId, String studentName) {
    this.studentId = studentId;
    this.studentName = studentName;
}

// Static method to display attendance policy
public static void showAttendancePolicy() {
    System.out.println("Attendance Policy:");
    System.out.println("Minimum 75% attendance is required to appear for exams.");
}

// Method to pass current object using this keyword
public void registerAttendance(int totalClasses, int attendedClasses) {
    Attendance attendance = new Attendance(this, totalClasses, attendedClasses);
    attendance.displayAttendance();
}

// Inner class for attendance details
class Attendance {

    private Student student; // Reference to outer class object
    private int totalClasses;
    private int attendedClasses;

    public Attendance(Student student, int totalClasses, int attendedClasses) {
        this.student = student; // receiving current object
        this.totalClasses = totalClasses;
        this.attendedClasses = attendedClasses;
    }

    public void displayAttendance() {
        double percentage = (attendedClasses * 100.0) / totalClasses;

        System.out.println("Student ID : " + student.studentId);
        System.out.println("Student Name : " + student.studentName);
        System.out.println("Attendance % : " + percentage);
    }
}

}

public class AttendanceTrackingSystem {
    public static void main(String[] args) {

        Student.showAttendancePolicy();
        System.out.println();

        Student s1 = new Student(101, "Rahul");
        s1.registerAttendance(100, 80);

        System.out.println();
    }
}

```

```

        // Object reference reassignment
        Student s2 = new Student(102, "Anita");
        s1 = s2; // s1 now refers to Anita's object
        s1.registerAttendance(90, 60);
    }
}

```

Output:

Attendance Policy:

Minimum 75% attendance is required to appear for exams.

Student ID : 101

Student Name : Rahul

Attendance % : 80.0

Student ID : 102

Student Name : Anita

Attendance % : 66.66666666666667

13. Insurance Policy Management

Design a Policy class.

Tasks:

- Use constructor overloading for different policy types
- Use final method for premium calculation
- Use static variable for company name
- Implement strict access control

```
class Policy {
```

```

    // Static variable (common company name)
    private static String companyName = "SecureLife Insurance";

```

```

    // Private variables (strict access control)
    private int policyId;
    private String policyHolderName;
    private String policyType;
    private double sumAssured;

```

```

    // Constructor for Basic Policy
    public Policy(int policyId, String policyHolderName, double sumAssured) {
        this.policyId = policyId;
        this.policyHolderName = policyHolderName;
        this.sumAssured = sumAssured;
    }

```

```

        this.policyType = "Basic";
    }

    // Overloaded Constructor for Premium Policy
    public Policy(int policyId, String policyHolderName,
        double sumAssured, String policyType) {
        this.policyId = policyId;
        this.policyHolderName = policyHolderName;
        this.sumAssured = sumAssured;
        this.policyType = policyType;
    }

    // Final method for premium calculation (cannot be overridden)
    public final double calculatePremium() {
        double rate;

        if (policyType.equalsIgnoreCase("Premium")) {
            rate = 0.08; // 8% premium rate
        } else {
            rate = 0.05; // 5% premium rate
        }

        return sumAssured * rate;
    }

    // Public method to display policy details
    public void display() {
        System.out.println("Company Name    : " + companyName);
        System.out.println("Policy ID      : " + policyId);
        System.out.println("Policy Holder  : " + policyHolderName);
        System.out.println("Policy Type    : " + policyType);
        System.out.println("Sum Assured    : ₹" + sumAssured);
        System.out.println("Premium Amount : ₹" + calculatePremium());
    }
}

public class InsurancePolicyManagement {
    public static void main(String[] args) {

        Policy p1 = new Policy(101, "Rahul", 500000);
        Policy p2 = new Policy(102, "Anita", 800000, "Premium");

        p1.display();
        System.out.println();

        p2.display();
    }
}

```

Output:

Company Name : SecureLife Insurance
Policy ID : 101
Policy Holder : Rahul
Policy Type : Basic
Sum Assured : ₹500000.0
Premium Amount : ₹25000.0

Company Name : SecureLife Insurance
Policy ID : 102
Policy Holder : Anita
Policy Type : Premium
Sum Assured : ₹800000.0
Premium Amount : ₹64000.0

14. Hotel Room Booking System

Create a Room class.

Requirements:

- Use static nested class for hotel services
- Overload booking methods for hourly and daily stay
- Use access control for room availability
- Use this keyword for object identification

```
class Room {  
  
    private int roomNumber;  
    private String roomType;  
    private boolean available; // Access controlled  
  
    // Static nested class for hotel services  
    static class HotelServices {  
        public static void showServices() {  
            System.out.println("Hotel Services:");  
            System.out.println("1. Free WiFi");  
            System.out.println("2. 24/7 Room Service");  
            System.out.println("3. Complimentary Breakfast");  
        }  
    }  
  
    // Constructor  
    public Room(int roomNumber, String roomType, boolean available) {  
        this.roomNumber = roomNumber; // using this keyword  
        this.roomType = roomType;  
        this.available = available;  
    }  
}
```

```

// Getter for availability (controlled access)
public boolean isAvailable() {
    return available;
}

// Overloaded method for hourly booking
public void bookRoom(int hours) {
    if (this.available) { // object identification using this
        this.available = false;
        double cost = hours * 200; // ₹200 per hour
        System.out.println("Room " + this.roomNumber +
            " booked for " + hours + " hours.");
        System.out.println("Total Cost: ₹" + cost);
    } else {
        System.out.println("Room not available.");
    }
}

// Overloaded method for daily booking
public void bookRoom(int days, boolean dailyStay) {
    if (this.available && dailyStay) {
        this.available = false;
        double cost = days * 1500; // ₹1500 per day
        System.out.println("Room " + this.roomNumber +
            " booked for " + days + " days.");
        System.out.println("Total Cost: ₹" + cost);
    } else {
        System.out.println("Room not available.");
    }
}

public void display() {
    System.out.println("Room Number : " + roomNumber);
    System.out.println("Room Type : " + roomType);
    System.out.println("Available : " + available);
}
}

public class HotelBookingSystem {
    public static void main(String[] args) {

        // Access static nested class
        Room.HotelServices.showServices();
        System.out.println();

        Room r1 = new Room(101, "Deluxe", true);
        Room r2 = new Room(102, "Suite", true);

        r1.display();
    }
}

```

```

        r1.bookRoom(5); // Hourly booking
        System.out.println();

        r2.display();
        r2.bookRoom(2, true); // Daily booking
    }
}

```

Output:

Hotel Services:

1. Free WiFi
2. 24/7 Room Service
3. Complimentary Breakfast

Room Number : 101

Room Type : Deluxe

Available : true

Room 101 booked for 5 hours.

Total Cost: ₹1000.0

Room Number : 102

Room Type : Suite

Available : true

Room 102 booked for 2 days.

Total Cost: ₹3000.0

15. E-Learning Platform Course Module

Design a CourseModule class.

Tasks:

- Use inner class for lessons
- Use static variable for platform name
- Use final variable for course ID
- Implement constructor chaining using this()

```

class CourseModule {

    // Static variable for platform name
    static String platformName = "LearnX Online Platform";

    // Final variable for Course ID
    private final String courseId;

    private String courseName;

```

```

// Default Constructor (Constructor Chaining)
public CourseModule() {
    this("C001", "Default Course"); // calling parameterized constructor
}

// Parameterized Constructor
public CourseModule(String courseId, String courseName) {
    this.courseId = courseId; // final variable initialized
    this.courseName = courseName;
}

// Inner class for Lessons
class Lesson {
    private String lessonTitle;
    private int duration; // in minutes

    public Lesson(String lessonTitle, int duration) {
        this.lessonTitle = lessonTitle;
        this.duration = duration;
    }

    public void displayLesson() {
        System.out.println("Lesson Title : " + lessonTitle);
        System.out.println("Duration    : " + duration + " minutes");
    }
}

public void displayCourse() {
    System.out.println("Platform Name : " + platformName);
    System.out.println("Course ID    : " + courseId);
    System.out.println("Course Name  : " + courseName);
}
}

public class ELearningPlatform {
    public static void main(String[] args) {

        // Using parameterized constructor
        CourseModule course1 = new CourseModule("C101", "Java Programming");

        course1.displayCourse();

        // Creating inner class object
        CourseModule.Lesson lesson1 = course1.new Lesson("OOP Concepts", 60);

        lesson1.displayLesson();

        System.out.println();

        // Using default constructor (constructor chaining)

```

```
        CourseModule course2 = new CourseModule();  
        course2.displayCourse();  
    }  
}
```

Output:

Platform Name : LearnX Online Platform
Course ID : C101
Course Name : Java Programming
Lesson Title : OOP Concepts
Duration : 60 minutes

Platform Name : LearnX Online Platform
Course ID : C001
Course Name : Default Course
